RESULT

JPX Model Training and Backtest Analysis

CNN_ENSEMBLE | 21-day window | 17 validation intervals | Based primarily on this training.log

Positive backtest results, but not enough to establish stable predictive ability.

+28.54%	14 / 17	0.00395
Direct sum of daily returns	Positive-return validation intervals	Simple mean of interval Rank ICs

Training completed and the model was saved. Total time: 1 hour 58 minutes 50 seconds; CUDA was used, with 106 epochs run in total. [1]

01 Return distribution: mostly positive, but still concentrated

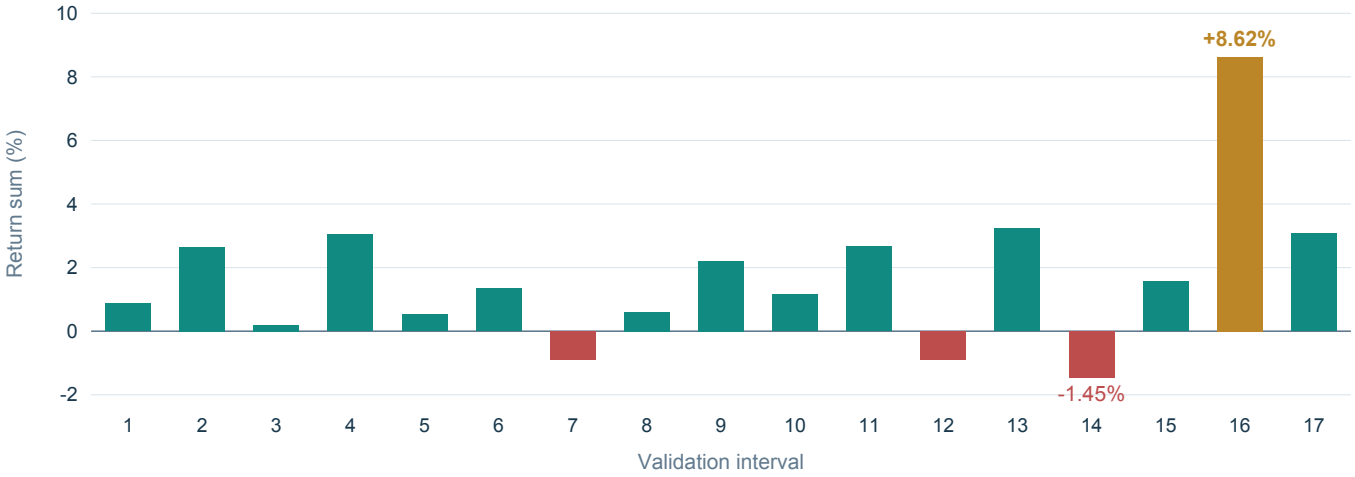


Figure 1 | Sum of daily returns for each validation interval; interval 16 is highlighted in gold. This is neither a daily equity curve nor a per-trade win rate.

Positive signal: 14 intervals had positive returns. Interval 16 contributed +8.62 percentage points, about 30.2% of the total return sum. Even excluding this best interval, the remaining intervals totaled +19.92%. [1]

Main limitation: Rank IC was positive in 12 intervals and negative in 5, with a simple mean of only 0.00395 and a median of 0.00499. The direction is slightly positive, but the magnitude is small. The log cannot establish statistical significance or an advantage after costs. [1]

Current assessment: Keep this as a baseline experiment. Next, check whether predictions have lost cross-sectional variation, verify evaluation definitions and early-stopping rules, and only then decide whether to enlarge the model.

28.54% is the sum of Target-weighted returns as defined by the code, before transaction costs. It is neither a compounded nor an annualized return.

Complete Results for 17 Intervals

Training and validation values are retained; repeated Stage/epoch startup messages and result dictionaries are omitted. [1]

Interval	Epochs	Mean training loss	Validation loss	Sharpe	Rank IC	Return sum
1	9	0.000504	0.000184	0.0688	0.0094	+0.88%
2	7	0.000199	0.000176	0.3153	0.0112	+2.65%
3	6	0.000173	0.000194	0.0225	0.0050	+0.20%
4	6	0.000181	0.000304	0.2864	0.0126	+3.04%
5	6	0.000208	0.000216	0.0427	-0.0016	+0.53%
6	6	0.000207	0.000253	0.1158	0.0003	+1.35%
7	6	0.000212	0.000396	-0.0400	-0.0142	-0.90%
8	6	0.000231	0.000307	0.0523	-0.0025	+0.60%
9	6	0.000242	0.000254	0.1413	0.0020	+2.21%
10	6	0.000244	0.000214	0.0652	0.0054	+1.15%
11	6	0.000241	0.000183	0.3235	0.0072	+2.67%
12	6	0.000236	0.000425	-0.0453	-0.0213	-0.89%
13	6	0.000248	0.000642	0.0964	0.0114	+3.23%
14	6	0.000277	0.000315	-0.0503	-0.0085	-1.45%
15	6	0.000281	0.000306	0.0564	0.0010	+1.56%
16	6	0.000283	0.000261	0.3220	0.0369	+8.62%
17	6	0.000281	0.000234	0.1151	0.0130	+3.08%

02 How to read the metrics

Loss: This run uses HuberLoss in model_3. These values are not MAE or direct return percentages. The training column is the sample-weighted mean across all epochs actually run, not the last epoch's loss. [2][3]

Sharpe: The supplied evaluation code uses mean daily return divided by the sample standard deviation of daily returns, without an annualization factor. Do not treat these figures as annualized Sharpe, or average interval Sharpes to obtain the full-period Sharpe. [2]

Rank IC: Spearman correlation between predictions and Target is computed each day and then averaged within an interval. The report's 0.00395 is the equal-weighted mean of the 17 interval values, not the full-period IC weighted by valid trading days. [1][2]

Returns: Each day, the code goes long 200 stocks and short 200 stocks; long weights total +0.5 and short weights -0.5. This is a Target-weighted strategy return proxy. It has no model for execution prices, slippage, fees, or stock-borrowing costs. [2]

Address These Four Issues First

Log facts, code behavior, and hypotheses to test are distinguished below. Warnings are not treated as proof of model failure.

01 Locate constant-input warnings by date

Known: Interval 15 produced a ConstantInputWarning. Spearman correlation is undefined for constant arrays and returns NaN; the final mean() used for IC currently excludes NaN by default. [1][2][4][5]

Not yet known: The log does not reveal whether Prediction or Target was constant, or how many days were affected. If predictions are tied, stock rankings may depend on how ties are sorted. Output daily std and nunique for both arrays, valid IC day counts, and NaN dates instead of merely suppressing the warning.

02 The early-stopping threshold may be too coarse

It is not yet possible to conclude that stopping was too early. Known settings: min_change=0.00001 and patience=5. All 17 intervals triggered early stopping; 15 stopped exactly at epoch 6. For loss values of 0.00020 to 0.00028, the threshold is about 3.6% to 5.0%. [1][2]

Interpretation: Improvements in multiple intervals did not exceed the threshold, so the patience counter was not reset. This setting may overlook small improvements, but running longer does not guarantee better validation performance. Create a separate, chronologically ordered validation set within the training period, monitor it, and save the best weights. Reserve the outer backtest intervals for evaluation.

03 Mean training loss is not the final model's error

For interval 1, mean training loss was 0.000504, final-epoch loss was 0.00016050, and validation loss was 0.00018445. The first value mixes different training stages; comparing it directly with the final model's validation loss can lead to a mistaken assessment of generalization. [1][2]

The current early-stopping logic also does not restore the best epoch's weights. Record final-epoch loss, the best inner-validation loss, and the epoch actually used. Recompute training and validation errors under the same eval mode.

04 The 17 intervals are not 17 independent models

The updated main workflow creates the model and Adam outside the interval loop, carrying weights and optimizer state across intervals. However, standardization parameters are re-estimated for each interval. This is one continuously updated training path, not 17 independent experiments. [2]

Re-standardization changes the input scale of the same feature, so the effect of carrying weights forward needs a controlled comparison. Retain this run as a baseline and compare reinitializing each interval with continued training. Do not change architecture, loss, and data splitting at the same time.

Additional note: HuberLoss defaults to delta=1. If most prediction errors are smaller than 1, it mainly uses the half-squared-error branch. Check the error distribution; do not assume it already handles tail risk effectively simply because its name includes Huber. [3]

Next Experiments and Evidence Needed

Make the results reproducible and the metrics verifiable before pursuing higher returns or more complex models.

03 Recommended execution order

Priority 1 | Complete the daily outputs. For each interval, save Date, SecuritiesCode, Prediction, Target, Weight, daily_return, and daily IC. Also save interval dates, scaler, model weights, optimizer state, parameters, and random seeds. Summarize constant predictions and missing IC values.

Priority 2 | Verify full-period performance. Recalculate Sharpe, valid-day counts, and drawdowns from non-overlapping out-of-sample daily returns. First verify the holding period represented by Target, signal availability, and trade timing; then build an equity curve after costs. Do not treat 28.54% as compounded, annualized, or executable net return.

Priority 3 | Establish minimum comparison baselines. Use zero predictions to compare Huber loss and MAE. For ranking strategies, use a simple linear model and same-day shuffled rankings as controls. All zero predictions are tied, so they are not a meaningful stock-ranking baseline.

Priority 4 | Change only one item at a time. First compare early-stopping thresholds and saving the best weights within the training period; then compare whether to reset the model across intervals. Fix the data splits and repeat runs with multiple random seeds. Avoid selecting only the best run.

Reproducibility limitation: This run only records that the final model was saved at checkpoints/CNN_ENSEMBLE. Without saved weights or out-of-sample predictions from earlier intervals, the final model cannot reconstruct past predictions as they were made at the time. Complete daily analysis requires rerunning and retaining the outputs listed above.

04 What this record can and cannot establish

Confirmed: The run completed; the interval metrics and early-stopping counts; the 28.54% sum of daily returns; one constant-input warning; and the final-model save message.

Not yet confirmed: Full-period annualized Sharpe, CAGR, maximum drawdown, performance after costs, statistical significance, or stability across random seeds. The 17 intervals are also not independent samples because training continues across them.

Data and method sources

[1] This training.log; Kaggle version 360031337. Values were parsed and recalculated from the original log.

[2] evaluation.py, deep_learning(1).py, and the updated JPX_deep_learning_models.py supplied in this conversation. Method analysis follows their code logic; no separate, complete byte-for-byte audit of the cloud files was performed.

[3] PyTorch official documentation: HuberLoss (delta and the piecewise loss definition).

[4] SciPy official documentation: spearmanr (constant input returns NaN).

[5] pandas official documentation: Series.mean (skipna=True).

Sources accessed: 2026-10-19. Intervals are numbered in log order. The log did not list actual interval dates, so none have been added.